

Exercice 2.3 — Agrégation multi-cubes

● Avancé

🕒 ~15 min live

Processus : `PLConsolide_Load_SupplyChain` — Serveur : `24Retail`

Structures des cubes confirmées live

Cube	Dimensions (ordre exact)
Supply Chain	<code>organization</code> , <code>Channel</code> , <code>product</code> , <code>Month</code> , <code>Year</code> , <code>Supply Chain Measures</code> , <code>Version</code>
Income Statement	<code>Currency Calc</code> , <code>organization</code> , <code>Year</code> , <code>Month</code> , <code>Account</code> , <code>Version</code>

Aspect	Valeur
Dimensions communes	<code>organization</code> , <code>Month</code> , <code>Year</code> , <code>Version</code>
Mesure SC lue	<code>Total Cost of Goods Sold – Regional</code> via <code>Channel Total</code> / <code>Product Total</code>
Compte IS cible initial	<code>5999</code> (Cost of Sales) → ✗ protégé par règle
Compte IS cible v2	<code>6130</code> (Other Expense) → ✗ aussi protégé
Architecture IS	Cube de reporting calculé — toutes cellules gouvernées par règles TM1

⚠ Découverte architecturale critique

Income Statement est un cube entièrement calculé par règles TM1
Toute tentative de `CellPutN` retourne :
`Rule applies to cell (error repeats 119 times)`

Le cube Income Statement de 24Retail agrège depuis Revenue et Supply Chain via son moteur de règles. Il n'est pas conçu pour recevoir des écritures directes via processus TI.

Étapes d'exploration live

- 1 **Lecture des structures des deux cubes via MCP**
`get_cube_dimensions` + `get_cube_sample_members` sur Supply Chain et Income Statement. Identification des dimensions communes et des mesures disponibles.
- 2 **Génération et déploiement v1**
Compte cible `5999` → Syntaxe valide  → Exécution → **Rule applies to cell**
- 3 **Investigation MDX sur Income Statement**
Vérification des comptes `6xxx` → tous ont des valeurs → tous protégés par règles TM1.
- 4 **Redéploiement v2 avec pAccount paramétrable**
Compte cible `6130` → **Rule applies to cell** — même résultat.
- 5 **Conclusion documentée**
Le processus est syntaxiquement valide et lit correctement Supply Chain. L'écriture dans Income Statement nécessite un compte non protégé ou un cube intermédiaire — à identifier avec le client.

Paramètres déployés

Paramètre	Type	Défaut	Description
<code>pVersion</code>	String	Forecast	Version à traiter
<code>pYear</code>	String	Y1	Année à traiter
<code>pCurrCalc</code>	String	Local	Currency Calc dans Income Statement

Paramètre	Type	Défaut	Description
pAccount	String	6130	Compte cible (à adapter — tous protégés sur 24Retail)

Code déployé (Epilog — extrait clé)

```
# Dimensions Supply Chain : organization, Channel, product, Month, Year, Supply Chain Measures,
Version
# Channel Total + Product Total = agregation toutes lignes
vCost = CellGetN( 'Supply Chain', vOrg, 'Channel Total', 'Product Total',
                  vMonth, pYear, 'Total Cost of Goods Sold - Regional', pVersion );

# Dimensions Income Statement : Currency Calc, organization, Year, Month, Account, Version
CellPutN( vCost, 'Income Statement', pCurrCalc, vOrg, pYear, vMonth, pAccount, pVersion );
# ⚠ Echoue sur 24Retail : Rule applies to cell (tous comptes proteges par regles TM1)
```

Résultats des runs

Run	pAccount	Statut	Erreur
v1 — compte fixe	5999	❌ Échec	Rule applies to cell × 120
v2 — compte paramétrable	6130	❌ Échec	Rule applies to cell × 120
Lecture SC seule	—	✅ OK	CellGetN fonctionne correctement

Bilan comparatif

15 min

Avec Bob (MCP)

3–6 h

Sans Bob

2

Découvertes architecturales

4

Paramètres déployés

Leçon clé de cet exercice

Sur un modèle PA mature comme 24Retail, les cubes de reporting (Income Statement) sont souvent entièrement calculés par règles TM1 et ne peuvent pas recevoir d'écritures directes. Bob détecte et diagnostique cette architecture en quelques minutes via MCP — ce qui éviterait des heures de débogage à un développeur sans connaissance préalable du modèle.